

19 Number of executors and dynamic resource allocation

1. What is an executor

An executor is a JVM process that runs tasks in parallel on a worker node.

Each executor has **its own cores and its own memory**.

👉 Memory assigned to an executor is **not shared** with other executors.

2. Two ways to place executors on a node

Option 1: One executor per node

This is called a **fat executor**.

Option 2: Multiple executors per node

This is called **thin executors**.

👉 The key difference is **how CPU and memory are divided across JVMs**.

3. Fat executor (1 executor per node – all cores and most memory)

Example:

- 1 executor uses almost all available cores
- 1 executor uses most of the node memory

Pros

- Simple to configure
- Fewer executors to manage

Cons

- One very large JVM
- **Very large heap → long GC pauses**
- One executor failure loses **all memory and all running tasks**
- One slow task can block many cores
- Poor fault isolation
- Large heap sizes (40–60 GB) cause:
 - Long stop the world GC pauses
 - Unpredictable task latency
- Shuffle, cache, and broadcast all compete for the same heap

👉 **Not recommended for most workloads**

Example: Fat executor

Let say available hardware:

- 6 Nodes
- 16 Cores per node
- 64 GB Memory per node

Reserve memory for:

- OS
- Hadoop daemons
- JVM overhead

Usable memory per node ≈ **58–60 GB**

With fat executors:

- 1 executor per node
- 15 cores
- ~58 GB executor memory

Total:

- 6 executors
 - Very large JVMs ❌
-

4. Thin executors (multiple executors per node)

Example:

- Multiple executors per node
- Each executor has fewer cores and smaller memory

Pros

- Smaller JVM heaps → **faster GC**
- Better fault isolation
- Failures affect fewer tasks
- More predictable performance
- Better memory management

Cons

- More executors to manage
- Slightly higher scheduling overhead

👉 Thin executors **do not increase total CPU**,
but they **reduce the blast radius of memory pressure and GC pauses**.

Memory benefit of thin executors (important)

Instead of:

- 1 executor with 58 GB heap ❌

You get:

- 3 executors with ~18–19 GB heap each ✅

Smaller heaps:

- GC runs faster
- Less memory fragmentation
- Less risk of OOM killing the whole node

👉 **Recommended approach**

A) Very thin executors (1 executor per core)

Example:

- One executor per core
- Each executor has **1 core**
- Many small JVMs on the same node

Example

Hardware:

- 16 cores per node

Very thin setup:

- 15 executors per node
- 1 core per executor
- Very small memory per executor

Pros

- Maximum fault isolation
- One executor failure affects only **one task**
- Small JVM heaps
- GC is usually fast

Problems with very thin executors (important)

- **Too many JVMs**
- High JVM startup overhead
- High memory overhead (each JVM needs its own heap + overhead)
- Inefficient use of memory
- Excessive task scheduling and context switching
- Driver becomes busy managing executors instead of work

👉 You waste memory and CPU on **managing executors**, not on processing data.

B) Ideal cores per executor (rule of thumb)

| 4–5 cores per executor is ideal

Why:

- Enough CPU parallelism
 - JVM stays manageable
 - GC remains efficient
 - Tasks stay responsive
-

6. How much memory per executor (rule of thumb)

| $\text{Executor memory} = (\text{Node memory} - \text{reserved memory}) / \text{executors per node}$

Always reserve memory for:

- OS and system processes
- Spark overhead (off heap, metadata)
- JVM overhead

Rule of thumb:

- Reserve **5–8 GB per node**
 - Also leave headroom for memory overhead
-

7. Full example with memory + cores

Hardware

- 6 Nodes
- 16 Cores per node
- 64 GB Memory per node

Step 1: Reserve resources

- 1 core for OS → usable cores = 15

- Reserve ~6 GB memory → usable memory ≈ 58 GB

Step 2: Choose cores per executor

- Cores per executor = 5

Executors per node = $15 / 5 = 3$

Step 3: Memory per executor

Memory per executor = $58 \text{ GB} / 3 \approx 19 \text{ GB}$

Recommended:

- `spark.executor.memory = 18g`
- `spark.executor.memoryOverhead = 2g`

Result per node

- 3 executors
- 5 cores each
- ~18 GB heap each
- Small, healthy JVMs 

9. Cluster wide view

If:

- Nodes = 6
- Executors per node = 3

Total executors = $6 \times 3 = 18$

Total cluster capacity:

- 18 executors
- 90 total cores

- ~324 GB executor memory

10. Passing configs in SparkSession

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .master("spark://spark-master:7077")
    .appName("executors")
    .config("spark.executor.cores", "2")
    .config("spark.executor.memory", "1g")
    .config("spark.cores.max", "4")
    .getOrCreate()
)
```

CPU decides how many tasks can run in parallel, but memory decides how stable and predictable Spark execution will be.

Dynamic Resource Allocation

What problem does it solve?

Until now, we assumed:

- Number of executors is fixed
- Resources are reserved even when Spark is idle

This causes two issues:

- Idle executors waste memory and CPU
- One job can block resources needed by another job

Dynamic allocation solves this by **adding and removing executors at runtime**.

What is dynamic allocation?

Dynamic allocation allows Spark to dynamically scale the number of executors based on workload demand.

Spark:

- requests more executors when there is pending work
- releases executors when they stay idle

How dynamic allocation works (high level)

1. Spark starts with **few or no executors**
2. Tasks get queued
3. Spark requests more executors from cluster manager
4. Executors are added
5. When executors stay idle for some time, they are removed

Resource Allocation Policy

At a high level, Spark should relinquish executors when they are no longer used and acquire executors when they are needed. Since there is no definitive way to predict whether an executor that is about to be removed will run a task in the near future, or whether a new executor that is about to be added will actually be idle, we need a set of heuristics to determine when to remove and request executors.

Request Policy

A Spark application with dynamic allocation enabled requests additional executors when it has pending tasks waiting to be scheduled. This condition necessarily implies that the existing set of executors is insufficient to simultaneously saturate all tasks that have been submitted but not yet finished.

Spark requests executors in rounds. The actual request is triggered when there have been pending tasks

for `spark.dynamicAllocation.schedulerBacklogTimeout` seconds, and then triggered again every `spark.dynamicAllocation.sustainedSchedulerBacklogTimeout` seconds thereafter if the queue of pending tasks persists. Additionally, the number of executors requested in each round increases exponentially from the previous

round. For instance, an application will add 1 executor in the first round, and then 2, 4, 8 and so on executors in the subsequent rounds.

The motivation for an exponential increase policy is twofold. First, an application should request executors cautiously in the beginning in case it turns out that only a few additional executors is sufficient. This echoes the justification for TCP slow start. Second, the application should be able to ramp up its resource usage in a timely manner in case it turns out that many executors are actually needed.

Graceful Decommission of Executors

Before dynamic allocation, if a Spark executor exits when the associated application has also exited then all state associated with the executor is no longer needed and can be safely discarded. With dynamic allocation, however, the application is still running when an executor is explicitly removed. If the application attempts to access state stored in or written by the executor, it will have to perform a recompute the state. Thus, Spark needs a mechanism to decommission an executor gracefully by preserving its state before removing it.

This requirement is especially important for shuffles. During a shuffle, the Spark executor first writes its own map outputs locally to disk, and then acts as the server for those files when other executors attempt to fetch them. In the event of stragglers, which are tasks that run for much longer than their peers, dynamic allocation may remove an executor before the shuffle completes, in which case the shuffle files written by that executor must be recomputed unnecessarily.

The solution for preserving shuffle files is to use an external shuffle service, also introduced in Spark 1.2. This service refers to a long-running process that runs on each node of your cluster independently of your Spark applications and their executors. If the service is enabled, Spark executors will fetch shuffle files from the service instead of from each other. This means any shuffle state written by an executor may continue to be served beyond the executor's lifetime.

In addition to writing shuffle files, executors also cache data either on disk or in memory. When an executor is removed, however, all cached data will no longer

be accessible. To mitigate this, by default executors containing cached data are never removed. You can configure this behavior with `spark.dynamicAllocation.cachedExecutorIdleTimeout`. When set `spark.shuffle.service.fetch.rdd.enabled` to `true`, Spark can use ExternalShuffleService for fetching disk persisted RDD blocks. In case of dynamic allocation if this feature is enabled executors having only disk persisted blocks are considered idle after `spark.dynamicAllocation.executorIdleTimeout` and will be released accordingly. In future releases, the cached data may be preserved through an off-heap storage similar in spirit to how shuffle files are preserved through the external shuffle service.

Key components involved

Dynamic allocation depends on **shuffle safety**.

Spark uses:

- **External Shuffle Service**
- OR
- **Shuffle tracking** (Spark 3+)

Without this, Spark cannot safely remove executors.

Minimum, maximum, and initial executors

Dynamic allocation is controlled by three main parameters:

- `spark.dynamicAllocation.initialExecutors`
- `spark.dynamicAllocation.minExecutors`
- `spark.dynamicAllocation.maxExecutors`

Example

```
minExecutors = 2
initialExecutors = 4
maxExecutors = 20
```

Meaning:

- Spark starts with 4 executors
 - Can scale down to 2
 - Can scale up to 20 if needed
-

When does Spark add executors?

Spark adds executors when:

- Tasks are waiting to be scheduled
- Current executors are fully utilized
- Backlog duration exceeds threshold

This is measured internally using **scheduler backlog**.

When does Spark remove executors?

Executors are removed when:

- They are idle for a configured time
- No active tasks
- Shuffle data is safe

Idle timeout configs:

- `spark.dynamicAllocation.executorIdleTimeout`
 - `spark.dynamicAllocation.cachedExecutorIdleTimeout`
-

Dynamic allocation and memory

Important implication:

- Memory is **not fixed anymore**
- Executors come and go
- Cached data may be lost if executor is removed

That's why:

- Dynamic allocation works best when caching is minimal
 - Or when shuffle tracking is enabled
-

Dynamic allocation vs fat thin executors

Dynamic allocation decides:

- **how many executors**

Fat vs thin decides:

- **how big each executor is**

They solve **different problems** and work together.

Recommended setup with dynamic allocation

Use:

- **Thin executors**
- **4–5 cores per executor**
- **Dynamic allocation enabled**

Avoid:

- Very thin executors
 - Very large heaps
-

Example config (Standalone / YARN)

```
from pyspark.sql import SparkSession
spark = (SparkSession.
builder.
master("spark://spark-master:7077")
.config("spark.dynamicAllocation.enabled", "true")
.config("spark.dynamicAllocation.minExecutors", "1")
.config("spark.dynamicAllocation.initialExecutors",
"1")
.config("spark.dynamicAllocation.maxExecutors", "4")
.config("spark.dynamicAllocation.schedulerBacklogTimeou
t", "10s")
.config("spark.dynamicAllocation.sustainedSchedulerBack
logTimeout", "1s")
.config("spark.executor.cores", "1"))
```

```
.config("spark.executor.memory", "1g")  
.appName("dataframe").getOrCreate()  
)
```

When NOT to use dynamic allocation

Dynamic allocation is NOT ideal when:

- You rely heavily on cached DataFrames
- You want stable, predictable executor layout
- Low latency interactive workloads (executor churn hurts)

In such cases:

- Static executors + persistent SparkSession is better

One line summary

| Dynamic allocation optimizes cluster utilization, not individual job speed.

Mental model

- Static allocation = reserve seats in advance
- Dynamic allocation = book seats when people arrive and release them when they leave